Session Hijacking

Concepts, Techniques, Tools & Countermeasures

CEH Module 11 - Study Notes

Contents

1. Session Hijacking Concepts
2. Application-Level Session Hijacking
3. Network-Level Session Hijacking
4. Session Hijacking Tools
5. Countermeasures
6. Lab: Hijack a Session using Zed Attack Proxy (ZAP)
7. Lab: Intercept HTTP Traffic using Bettercap
8. Detecting Session Hijacking

Disclaimer: These notes are for authorized educational and certification-study purposes only (e.g., CEH exam prep / lab environments such as vLabs). Only perform these techniques against systems and networks you own or are explicitly authorized to test.

1. Session Hijacking Concepts

What is Session Hijacking?

Session hijacking is the exploitation of a valid computer session - a Session ID / session token used to authenticate a user to a remote server - to gain unauthorized access to information or services. The attacker takes over an already-authenticated session instead of having to steal/guess the original username and password.

Why Sessions Are Hijackable

- HTTP is a stateless protocol, so applications use a Session ID (cookie, URL parameter, or hidden form field) to track a logged-in user
- If the Session ID can be predicted, stolen, or reused, an attacker can impersonate the victim without credentials
- Session IDs transmitted over unencrypted channels (plain HTTP) can be sniffed directly off the wire

Key Session Hijacking Concepts

Term	Description
Session ID / Token	Unique identifier issued by the server after authentication to track a user's session
Spoofing vs Hijacking	Spoofing pretends to be another user from the start; hijacking takes over an existing, already-authen
Session Fixation	Attacker sets/forces a known Session ID on the victim before login, then uses that same ID after the
Session Sidejacking	Sniffing session cookies from unencrypted traffic, often on shared Wi-Fi/LAN
Cross-Site Scripting (XSS)	Injecting script into a trusted site to steal cookies/session tokens from victims who visit it

Session Hijacking Process (General Flow)

- Sniff / monitor traffic between the victim and the server to locate an active session
- Identify or predict a valid Session ID associated with that session
- Either take over the session mid-stream (desynchronize the victim) or replay the captured ID to impersonate the victim
- Perform actions on the hijacked session as if the attacker were the legitimate user

Types of Session Hijacking

- Application-Level Hijacking - targets the HTTP session/session tokens of a web application
- Network-Level Hijacking - targets the underlying TCP/IP session (sequence numbers, packets) rather than the application layer

2. Application-Level Session Hijacking

Application-level hijacking focuses on obtaining or manipulating the Session ID that a web application uses to track an authenticated user.

Session Sniffing

Capturing network traffic (e.g., with Wireshark) to intercept a session cookie sent in cleartext over HTTP.

Predictable Session Token ID

If session IDs are generated with weak/predictable algorithms (sequential numbers, timestamps), an attacker can guess a valid ID.

Man-in-the-Browser Attack

Malware (a malicious browser extension/trojan) intercepts and manipulates calls between the browser and its security mechanisms, often targeting banking sessions.

Client-Side Attacks (XSS, Malicious JavaScript/Trojans)

Injected script runs in the victim's browser and exfiltrates cookies/session tokens to the attacker.

Cross-Site Request Forgery (CSRF)

Tricks an authenticated victim's browser into submitting unwanted requests to a site where they're logged in.

Session Replay Attack

Captured valid session data/requests are replayed later to repeat an action or regain access.

Session Fixation

Attacker supplies the victim a known Session ID (e.g., via a crafted link) before login; once the victim authenticates, the attacker uses that same ID.

Proxy / CRIME / Session Donation Attacks

Various techniques that trick a victim into using a session controlled or known by the attacker.

3. Network-Level Session Hijacking

Network-level hijacking attacks the TCP/IP session itself - sequence numbers, packet flow, and routing - rather than application tokens. It provides information (like sequence numbers) that can also assist application-level attacks.

TCP/IP Hijacking

Attacker injects spoofed packets with the correct sequence number into an active TCP session, desynchronizing the two legitimate endpoints and taking over the conversation.

Blind Hijacking

Attacker cannot see the return traffic but blindly injects data/commands into a session by correctly guessing sequence numbers.

UDP Hijacking

Since UDP has no sequence numbers/handshake, an attacker can simply spoof a response to a UDP request if they can see or guess it first.

Man-in-the-Middle (MITM) Attack

Attacker positions between client and server (e.g., via ARP spoofing) and relays/alters traffic, capturing session details.

ARP Spoofing / ARP Poisoning

Sends forged ARP messages on a LAN to associate the attacker's MAC address with the victim's or gateway's IP, redirecting traffic through the attacker.

RST Hijacking

Attacker injects a spoofed TCP RST packet with a correct sequence number to tear down a victim's connection (often paired with session takeover or DoS).

IP Spoofing

Forging the source IP address of packets to impersonate a trusted host and bypass IP-based access controls.

Packet Sniffing

Passive capture of packets on a network segment (especially switched/unswitched LANs) to extract session identifiers or credentials.

TCP Session Hijacking - Key Steps

- Find/track an active TCP session between victim and server (sniff traffic)
- Predict or capture the correct sequence and acknowledgment numbers
- Desynchronize the legitimate client's connection (e.g., flood it or force it out of sync)
- Inject spoofed packets with correct sequence numbers to the server as the 'victim'
- Take over the session and continue the communication as the attacker

4. Session Hijacking Tools

Tool	Type / Layer	Purpose

Zed Attack Proxy (ZAP)	Application-level	Intercepting proxy; capture/replay/modify HTTP(S) requests and session tokens
Bettercap	Network-level	ARP spoofing, MITM positioning, live HTTP(S)/traffic interception and manipulat
Wireshark	Network-level	Packet capture/analysis; sniff session cookies and TCP sequence numbers
Burp Suite	Application-level	Intercepting proxy for inspecting/modifying session tokens and requests
Ettercap	Network-level	MITM toolkit supporting ARP poisoning, sniffing, and packet filtering
Hamster & Ferret	Application-level	Classic sidejacking tools - sniff and replay HTTP session cookies
Hunt	Network-level	Tool for TCP connection hijacking/sniffing on a LAN

Note: These are dual-use security tools. Legitimate teams use them for authorized penetration testing, security research, and classroom labs under a signed scope of work.

5. Countermeasures

Protect the Session Itself

- Use long, random, cryptographically strong Session IDs (avoid predictable/sequential tokens)
- Regenerate the Session ID after login (prevents session fixation)
- Set short session expiration/idle timeouts and enforce re-authentication for sensitive actions
- Bind sessions to additional attributes (IP address / user-agent) where feasible, with care for false positives

Encrypt Everything

- Enforce HTTPS/TLS site-wide (HSTS) so cookies and tokens cannot be sniffed in cleartext
- Mark cookies as Secure (HTTPS only) and HttpOnly (not accessible to JavaScript, mitigating XSS theft)
- Use SameSite cookie attributes to reduce CSRF risk
- Avoid putting Session IDs in URLs (they can leak via browser history, referrer headers, logs)

Network-Level Defenses

- Use switched networks with port security; monitor for ARP spoofing (dynamic ARP inspection)
- Deploy IDS/IPS to detect anomalous TCP sequence behavior or ARP poisoning
- Use VPNs on untrusted networks (public Wi-Fi) to prevent sidejacking
- Implement network segmentation to limit sniffing exposure

Application & User Practices

- Sanitize all input/output to prevent XSS (which can be used to steal cookies)
- Implement proper logout functionality that invalidates the session server-side
- Educate users to log out of sessions on shared/public computers
- Apply the principle of least privilege so a hijacked session has limited blast radius

6. Lab: Hijack a Session using Zed Attack Proxy (ZAP)

Objective

Use OWASP ZAP as an intercepting proxy to capture HTTP requests/responses between a browser and a target web application, identify the session cookie, and reuse/replay it to demonstrate session hijacking.

General Workflow

- Launch ZAP and let it start its local proxy (default 127.0.0.1:8080)
- Configure the browser (or use ZAP's built-in browser) to route traffic through the ZAP proxy
- Browse to the target web application and log in as a test user
- In ZAP's 'Sites' / 'History' tab, locate the authenticated request and inspect its headers
- Identify the Session ID / cookie value (e.g., JSESSIONID, PHPSESSID, or a custom auth token)
- Use 'Manual Request Editor' (or Replay) to resend a request with the captured session cookie from a different browser/session, confirming the server accepts it as the original authenticated user
- Optionally use the captured token to access restricted pages without re-authenticating

Key idea: if the application accepts the replayed cookie from an unrelated client/location without additional checks, the session can be hijacked purely by possessing that token.

7. Lab: Intercept HTTP Traffic using Bettercap

Objective

Use Bettercap to position as a man-in-the-middle on a local network (via ARP spoofing) and intercept unencrypted HTTP traffic between a victim machine and the gateway/web server, capturing session cookies and credentials in transit.

General Workflow

- Identify the target's IP and the network gateway IP (e.g., using netdiscover or arp-scan)
- Launch Bettercap specifying the network interface
- Run network discovery within Bettercap to list live hosts
- Enable ARP spoofing targeting the victim and the gateway so traffic routes through the attacker machine
- Enable the HTTP proxy/sniffer module to log HTTP requests, headers, and cookies in real time
- Monitor the captured output for Set-Cookie / Cookie headers containing session identifiers
- Stop the spoofing module cleanly to restore normal ARP tables when done

Example commands (illustrative):

```
sudo bettercap -iface eth0
> net.probe on
> net.show
> set arp.spoof.targets <victim-ip>
> arp.spoof on
> net.sniff on
> http.proxy on
```

This lab works only against cleartext HTTP; properly configured HTTPS traffic will appear encrypted and the raw session cookie will not be directly readable, which is exactly why HTTPS everywhere is a core countermeasure.

8. Detecting Session Hijacking

Detection focuses on spotting anomalies in how and where a session token is being used, and unusual network behavior associated with MITM or sequence-prediction attacks.

Manual / Application-Level Detection

- Flag a session that suddenly switches IP address, geolocation, or user-agent mid-session
- Detect multiple concurrent logins using the same Session ID from different locations
- Monitor for abrupt, out-of-pattern actions performed by an authenticated session (privilege escalation attempts, rapid data access)
- Log and alert on an unusually high rate of failed/invalid session token attempts (could indicate brute-force/prediction attempts)
- Watch for session tokens appearing in server logs, referrer headers, or third-party requests (sign of leakage)

Network-Level Detection

- Monitor for duplicate/conflicting ARP entries (sign of ARP spoofing) using tools like arpwatch
- Use IDS/IPS signatures for abnormal TCP sequence number jumps or unexpected RST/SYN patterns
- Detect promiscuous-mode network interfaces on the LAN (possible sniffing activity)
- Correlate NetFlow/packet capture data to spot a host suddenly relaying traffic for two other hosts (classic MITM pattern)
- Deploy Intrusion Detection Systems (e.g., Snort/Suricata) with rules tuned for session hijacking indicators

Best-Practice Response

- Immediately invalidate/expire the suspected session server-side upon detection
- Force re-authentication (and MFA, if available) for the affected account
- Investigate logs to determine scope (what the hijacked session accessed/modified)
- Patch the root cause - weak token generation, missing HTTPS, XSS flaw, ARP spoofing exposure, etc.

Important: All hands-on labs above (ZAP session replay, Bettercap ARP spoofing/HTTP interception) must only be performed in an authorized lab environment (e.g., CEH practice range, your own isolated test network/VMs) or against systems/networks you have explicit written permission to test. Unauthorized interception of network traffic or session hijacking is illegal in most jurisdictions.